

Manual completo

Power BI → un único artefacto → múltiples informes → PDF

Objetivo: mantener un solo artefacto de Claude con varios tipos de informe (Ejecutivo, Finanzas, KAM, etc.), usando Power BI como fuente de datos y conservando tipografías, estilos, layouts y componentes visuales.

Arquitectura: Power BI Service → modelo semántico → API Execute Queries → backend seguro → JSON normalizado → único artefacto → informe seleccionado → PDF.

Importante: no conviene importar el PBIX como mecanismo operativo de datos. Power BI Service y su modelo semántico deben ser la fuente de verdad. Microsoft sí permite exportar un PBIX por REST API, pero eso es una función de descarga del reporte, no la vía recomendada para alimentar tu generador.

1. Qué vas a construir

El usuario abre un único artefacto y selecciona **Tipo de informe:** Ejecutivo, Finanzas, KAM, Gerencia, etc. Luego selecciona período y filtros, pulsa **Actualizar datos** y el backend consulta Power BI. El artefacto recibe JSON y renderiza el layout correspondiente. Finalmente pulsa **Exportar PDF**.

Los datos y cálculos críticos deben venir de Power BI. El artefacto debe encargarse principalmente de presentación, layout y narrativa. Si una medida ya existe en Power BI, no conviene recalcularla en JavaScript.

2. Paso 1 — Abrir Power BI Service

Abrí Power BI Service e inicié sesión con tu cuenta corporativa.

Entrá al workspace donde está tu tablero y abrí el reporte que querés utilizar como fuente.

3. Paso 2 — Identificar Workspace ID y Report ID

Mirá la URL del navegador. Una URL típica contiene
/groups/{workspaceId}/.../reports/{reportId}/...

Copió ambos GUID en un bloc de notas. También identificá el nombre exacto del modelo semántico asociado al reporte.

4. Paso 3 — Verificar el modelo semántico

Dentro del workspace buscá el elemento **Modelo semántico / Semantic model** asociado al reporte. Necesitaremos su Dataset/Semantic Model ID.

La API de ejecución de consultas trabaja contra el dataset/modelo semántico y acepta consultas DAX.

5. Paso 4 — Verificar permisos y configuración

La API Execute Queries requiere permisos apropiados sobre el dataset. Microsoft documenta `Dataset.Read.All` o equivalente para esta API.

Para usar Service Principal, el administrador debe habilitar en Power BI el tenant setting **Allow service principals to use Power BI APIs**. Hay una limitación importante: Service Principals no son compatibles con datasets con RLS en este escenario ni con datasets con SSO habilitado.

Antes de construir todo, pedí a IT/Power BI Admin que confirme: 1) Execute Queries habilitado; 2) acceso Read + Build al modelo; 3) si existe RLS; 4) si existe SSO; 5) si permiten Service Principal.

6. Paso 5 — Mensaje para IT

Podés enviar este pedido:

“Necesito consultar mediante Power BI REST API un modelo semántico existente, únicamente en lectura, para alimentar una aplicación interna de generación de informes. Necesito confirmar que Execute Queries esté habilitado, que mi usuario tenga Read + Build sobre el modelo y, para la implementación definitiva, si está permitido utilizar un Service Principal. También necesito confirmar si el modelo utiliza RLS o SSO.”

7. Paso 6 — Crear el backend

No conectes directamente el frontend/artefacto con credenciales corporativas de Power BI. El backend será el intermediario.

En VS Code abrí una terminal y ejecuté:

```
mkdir powerbi-backend
cd powerbi-backend
npm init -y
npm install express dotenv axios
```

La estructura inicial puede ser: `server.js`, `.env`, `powerbi/auth.js`, `powerbi/client.js`, `powerbi/queries.js` y una carpeta `reports`.

8. Paso 7 — Crear la aplicación en Microsoft Entra

Entrá al centro de administración de Microsoft Entra. Ruta habitual: **Identity** → **Applications** → **App registrations** → **New registration**.

Nombre sugerido: PowerBI-Informe-Generator. Elegí cuentas del directorio organizacional solamente y registrá la aplicación.

En Overview copió **Application (client) ID** y **Directory (tenant) ID**.

9. Paso 8 — Crear Client Secret

En la aplicación: **Certificates & secrets** → **New client secret**. Creá un secreto según la política de tu empresa.

Copió inmediatamente el **Value**. No publiques el secreto, no lo pegues en el frontend y no lo guardes en GitHub.

10. Paso 9 — Variables de entorno

Creá un archivo `.env` en el backend:

```
TENANT_ID=...
CLIENT_ID=...
CLIENT_SECRET=...
POWERBI_WORKSPACE_ID=...
POWERBI_DATASET_ID=...
```

Agregá `.env` a `.gitignore`.

11. Paso 10 — Dar acceso en Power BI

El registro de la aplicación en Entra no le da automáticamente acceso al workspace. Si utilizás Service Principal, el administrador debe permitirlo y asignar el acceso correspondiente al workspace/modelo.

Usá el menor nivel de acceso que permita la consulta. No des permisos de edición si solo necesitás lectura.

12. Paso 11 — Primera prueba de API

Antes de construir los informes, probá solamente autenticación + workspace + dataset + una consulta DAX pequeña.

La API documentada para consultar un modelo en un workspace es POST
/v1.0/myorg/groups/{groupId}/datasets/{datasetId}/executeQueries.

Una consulta de prueba puede ser EVALUATE ROW("Prueba", 1). Después conviene probar una tabla o medida real del modelo.

13. Paso 12 — Diseñar consultas por informe

No descargues todo el modelo. Cada informe debe pedir solamente los datos que necesita.

Ejecutivo: KPIs, evolución Real vs BO, desvíos, EBITDA y margen. Finanzas: Real, BO, OPEX, provisiones, pendiente de facturar, Aging y DSO. KAM: cliente, facturación, costos y margen.

La API actualmente limita cada consulta a 100.000 filas o 1.000.000 de valores, 15 MB y 120 consultas por minuto por usuario. Diseñar consultas pequeñas evita llegar a estos límites.

14. Paso 13 — Usar medidas de Power BI como fuente de verdad

Si Power BI ya tiene [Real], [BO], [Variación], [Variación %], [EBITDA], etc., el artefacto debe consumir esas medidas.

No dupliques la lógica DAX en JavaScript salvo que exista una razón concreta. Esto evita diferencias entre el tablero y el PDF.

15. Paso 14 — Crear un JSON normalizado

El backend debe transformar las respuestas de Power BI en una estructura estable que el único artefacto pueda consumir.

Ejemplo:

```
{ reportType: "ejecutivo", filters: { periodo: "2026-07", sociedad: "IHSA" }, kpis: {  
  real: 125000000, bo: 118000000, variacion: 7000000, variacionPct: 5.93, ebitda:  
  32000000 }, evolucion: [], desvios: [], aging: [] }
```

16. Paso 15 — Diseñar un único artefacto con múltiples informes

El frontend debe tener un selector **Tipo de informe**. Los valores pueden ser Ejecutivo, Finanzas, KAM y Gerencia.

Internamente, mantené una configuración por tipo de informe. Ejemplo conceptual: reportTypes.ejecutivo, reportTypes.finanzas, reportTypes.kam.

No copies todo el HTML para cada informe. Reutilizá componentes: Header, KPI, KPIGrid, Chart, Table, Commentary, Footer y ReportLayout.

17. Paso 16 — Mantener formatos, tipografías y estilos

La tipografía, tamaños, colores, espaciados, bordes, tarjetas y componentes deben estar definidos en el código del artefacto.

Los datos provenientes de Power BI solo llenan esos componentes. Por lo tanto, actualizar datos no modifica el diseño.

Esto permite que los informes tengan una identidad visual consistente aunque muestren información diferente.

18. Paso 17 — Flujo de actualización

Usuario selecciona informe → período → sociedad → vertical → pulsa **Actualizar datos** → backend ejecuta DAX → backend devuelve JSON → artefacto actualiza componentes.

Mostrará siempre la fecha/hora de actualización y la fuente, por ejemplo: Fuente: Power BI – Real vs BO/OPEX.

19. Paso 18 — Exportar PDF

El botón **Exportar PDF** debe tomar el informe ya renderizado. No debe pedirle a Claude que vuelva a diseñar el documento desde cero.

El flujo recomendado es: datos Power BI → HTML/CSS fijo → render → PDF. Así la exportación mantiene la misma tipografía, espaciado y estructura.

20. Paso 19 — Agregar nuevos informes

Para agregar un nuevo informe no crees otro artefacto. Agregá una nueva configuración y las consultas DAX necesarias.

Ejemplo: ejecutivo, finanzas, kam, gerencia. El selector del mismo artefacto mostrará las nuevas opciones.

21. Paso 20 — Seguridad

Nunca coloques Client Secret, tokens de Microsoft ni credenciales corporativas en el frontend o en el código del artefacto.

No subas .env a GitHub. Usá variables de entorno y secretos del servicio donde alojes el backend.

No mandes el PBIX completo a Claude si no es necesario. Mandá únicamente el JSON mínimo necesario para cada informe.

22. Paso 21 — Arquitectura definitiva

Arquitectura final recomendada:

Power BI Service → Modelo semántico → Power BI REST API / Execute Queries → Backend seguro → JSON normalizado → UN ÚNICO ARTEFACTO → Ejecutivo / Finanzas / KAM / Gerencia → Exportar PDF

23. Paso 22 — Checklist de implementación

- Workspace ID identificado
- Report ID identificado
- Dataset/Semantic Model ID identificado
- Execute Queries permitido
- Read + Build confirmado
- RLS/SSO verificado
- Método de autenticación definido
- App Registration creada
- Secret almacenado de forma segura
- Service Principal autorizado si corresponde
- Primera consulta DAX funcionando
- JSON normalizado definido
- Informe Ejecutivo funcionando
- Informe Finanzas funcionando
- Informe KAM funcionando
- Exportación PDF funcionando
- Fecha/hora de actualización visible
- .env excluido de Git

24. Qué NO hacer

1. No importar el PBIX en cada generación.
2. No descargar todo el modelo.
3. No duplicar medidas DAX en JavaScript sin necesidad.
4. No poner secretos en el frontend.
5. No crear un artefacto diferente para cada informe.
6. No regenerar el diseño del PDF con IA en cada ejecución.
7. No asumir que Service Principal funciona si el modelo usa RLS/SSO; verifícalo primero.

25. Nota sobre PBIX

Power BI sí ofrece una REST API para exportar un reporte a PBIX/RDL, incluso desde un workspace. Sin embargo, para tu objetivo el PBIX debe considerarse un archivo de desarrollo/descarga, no la interfaz de datos del generador de informes.

La API de exportación de PBIX requiere permisos específicos y tiene limitaciones. Por eso la arquitectura recomendada sigue siendo consultar el modelo semántico directamente.

26. Fuentes oficiales

Microsoft Learn — Execute Queries:

<https://learn.microsoft.com/en-us/rest/api/power-bi/datasets/execute-queries>

Microsoft Learn — Execute Queries in Group:

<https://learn.microsoft.com/en-us/rest/api/power-bi/datasets/execute-queries-in-group>

Microsoft Learn — Export Report in Group:

<https://learn.microsoft.com/en-us/rest/api/power-bi/reports/export-report-in-group>

Microsoft Learn — Export To File in Group:

<https://learn.microsoft.com/en-us/rest/api/power-bi/reports/export-to-file-in-group>

Nota de implementación

Los nombres exactos de algunos menús pueden variar según el idioma, versión de Power BI y permisos de tu organización. La primera prueba debe ser siempre una consulta DAX mínima antes de construir los informes completos.

Documentación verificada en septiembre de 2026. La API Execute Queries permite ejecutar DAX contra datasets y documenta límites de filas/valores, tamaño y frecuencia. También documenta las restricciones relevantes de Service Principal con RLS/SSO. La exportación de PBIX desde un workspace está disponible mediante REST API, pero no es la arquitectura recomendada para este proyecto.